UUL Compass — 项目全景文档

接收人： David Wu（工程师） **撰写人：** Jerry Shi **日期：** 2026年4月1日 **代码仓库：** https://github.com/jerryshimax/uul-compass **当前状态：** Phase 1 前端 Demo 完成，需要 UI 设计升级 + Vercel 部署 + Supabase 接入

第一章：项目背景与战略意义

1.1 UUL Global 是什么

UUL Global 是一家全球端到端物流公司，前身是 2004 年成立的 US United Lines（美国第三大中美货代）。2026年4月1日，Jerry Shi 通过 Synergis Capital 完成对 UUL Global 的收购。

核心业务三大支柱：

支柱一：端到端物流（Source to Site）

- 国际货运代理：海运整箱/拼箱（FCL/LCL）、空运、铁路
- 项目货物与超重超大件运输
- 报关与贸易合规
- 美国国内运输：整车（TL）、零担（LTL）、特种车辆、末端配送
- 仓储与分销（通过子公司 LC Warehousing）
- 中国工厂级买方集货

支柱二：供应链金融（Move the Risk, Not Just the Freight）

- 与银行和金融科技合作的贸易融资
- 第三方信用承保（风险不在 UUL 资产负债表上）
- 项目制货运的付款条款结构化
- 与专业供应链金融提供商的推荐合作（如 Klear 专注太阳能、渣打银行上海）

支柱三：全球合规与采购（Your Bridge from East to West）

- 采购与买方代理服务：在海外供应商中寻找、审核、采购
- 贸易合规与关税分类
- IRA/国产含量合规咨询（对能源客户至关重要）
- 在中国主要港口城市的双语团队

目标行业：

- AI 基础设施与数据中心（变压器、开关柜、发电机、冷却系统）
- 可再生能源（太阳能板、逆变器、储能系统）
- 先进制造与传统能源（炼油厂设备、发电组件）

全球布局：

- 美国：Greensboro, NC（总部）
- 中国：深圳、上海、宁波、广州、苏州、青岛、天津
- 香港
- 越南：胡志明市
- 加拿大、比利时、德国、尼日利亚、墨西哥

1.2 关联公司（Sister Companies）

UUL Global 不是一个独立运营的公司，而是一个由多个关联实体组成的网络，正在进行整合：

实体	代码	说明
UUL Global	UUL	主体公司，美国总部，品牌重塑后的对外形象
美航集团（Meihang）	MH	前身 US United Lines 的中国运营主体，~50名员工，拥有大量海运合同和客户关系
星航（Star Navigation）	XH	大陆客户关系与现场执行，中国北方传统物流，华南（厦门、深圳）业务
Sage Line Logistics	SAGE	独立实体，互补运营

整合策略的关键： 当合并在一个母公司下时，Jerry 可以告诉承运商"我的销售代表从3人增加到50人"——承运商信用度直线上升。目前 UUL 无法可信地声称 3,000 TEU 的量，需要美航的量来支撑谈判。

重要：LC Warehousing（良仓）不是 Compass 的核心模块。 仓储按第三方供应商管理，LC 是 preferred vendor 之一，但 UUL 的核心是货运代理 + 合规 + 采购。LC 主要做电商 fulfillment，与 UUL 的 renewable energy/AIDC/logistics 定位有差异。

1.3 Compass 的战略定位

Compass 不只是一个项目管理工具。它是两件事：

第一：100 天并购后整合（PMI）追踪器

- 2026年4月1日是 Day 1，100天到2026年7月10日
- Board 成员（葛总、Billy、Season）、C-suite（Jason CEO、Josh COO、Serena CFO）、部门负责人都能看到并参与
- 每个部门的改变、落到谁头上、做什么事、什么时候做完——全部可视化

第二：UUL 的运营数据中枢（Operational OS）

- 统一所有办公室（US、中国、HK）的物流数据
- 打通从 origin（中国工厂）→ ocean freight → domestic logistics → drayage → delivery 的全链路数据
- 为 Pallet AI 报价系统提供底层结构化数据

1.4 终极愿景：AIOS（AI 物流操作系统）

UUL 正在与 **Pallet**（pallet.com）做 pilot。Pallet 是一个 AI 物流 workforce 平台，支持 15+ use cases：

1. 文档处理 — 消除各类文档的数据录入
2. **货运报价** — 快速报价赢得更多业务（当前 pilot，95% 完成）
3. 跟踪追踪 — 无摩擦沟通，更新货运状态
4. 数据录入自动化 — 处理来自各种来源的非结构化信息
5. 电子表格解析 — 从多种格式提取信息
6. 现货市场检查 — 集成定价数据用于报价决策
7. 里程计算 — 查询距离用于费率确定
8. 报价生成 — 基于自定义费率规则创建报价
9. 第三方门户导航 — 无需 API/EDI 访问货运平台
10. 客服自动化 — 处理日常沟通
11. 货运状态更新 — 维护实时追踪信息
12. 拼柜建议 — 自动化装柜合并
13. POD（签收证明）检索 — 提取交货确认
14. 发票审计 — 处理和验证发票
15. 海关处理 — 处理进出口文件

终极路线图：

```
Compass 结构化数据 → Pallet AI 全链路运营 → 90% AI 驱动的物流公司
```

100 天计划的核心目的就是为这个终极目标做准备：

1. **梳理 structural architecture** — 每个部门的能力、部门间合作机制、供应链机制
2. **量化 SOP + 提取 tribal knowledge** — 把葛总、Marco 等关键人物头脑中的隐性知识（承运商关系、定价逻辑、客户偏好、操作规程）结构化
3. **建数据库** — 所有运营数据结构化，让 AI 可以调用、查询、执行

第二章：核心团队与权限设计

2.1 首批用户（8人）

姓名	中文名	角色	系统权限	工作地点	关键说明
Jerry Shi	施嘉懋	Owner & Board Chair	全部管理员	亚洲/美国	项目发起人，最终决策者
Alic Ge	葛总	Board Director	全部	香港/亚洲	**只用手机，不用电脑**——这是最关键的设计约束。所有界面必须在手机上完美运行
Billy	大哥	Board Director	全部	—	Board 成员
Season	—	Board Director, Finance	全部	宁波	Board 成员，财务背景
Jason Likens	—	CEO, US Operations	全部	美国	2026年3月16日任命。美国国内物流专长，国际货运相对薄弱，需要亚洲团队支持
Josh Foster	—	COO, US Operations	全部	美国	2026年3月16日任命。运营专长，团队扩展，流程优化
Serena Lin	林静	CFO	全部（尤其财务模块）	—	新任 CFO
David Wu	—	Engineer	全部管理员	北美	负责部署、设计升级、技术实现

2.2 权限矩阵

系统采用 RBAC（基于角色的访问控制），每个用户有一个角色，决定能看到什么、能改什么：

角色	仪表盘	100天计划	货运	报价	财务	销售	运营	人员
Owner (Jerry)	全部	全部读写	全部	全部	全部	全部	全部	全部
Board (葛总, Billy, Season)	全部	全部读写	全部	全部	全部	全部	全部	全部
Executive (Jason, Josh, Serena)	全部	全部读写	全部	全部	全部*	全部	全部	全部
部门负责人	部门视图	部门读写	部门	部门	不可见	部门	部门	部门
普通成员	部门视图	自己的任务	分配的	分配的	不可见	自己的	部门	部门
只读查看	只读	只读	只读	不可见	不可见	不可见	只读	只读

*Serena（CFO）拥有财务模块完整权限

2.3 多实体数据隔离（Multi-Entity RLS）

数据库中的每张表都有 `entity_id` 字段，用于数据隔离：

- Jerry 可以看到所有实体（UUL + 美航 + 星航 + Sage Line）的数据
- Marco 可以看到 UUL + 美航的数据
- 某些用户只能看到自己所属实体的数据
- 财务表有额外限制：只有 owner / board / executive / finance 角色可见

通过 Supabase 的 Row Level Security（RLS）在数据库层面强制执行，不是应用层过滤。即使有人直接访问 API，也无法看到无权访问的数据。

2.4 登录方式

- **主要方式：Magic Link（邮箱链接登录）** — 用户输入邮箱，收到一封带登录链接的邮件，点击即登录。不需要记密码。
- **备用方式：Google SSO** — 给美国团队用（他们已经在 Google Workspace 上）
- **仅限邀请制** — 没有自助注册。Jerry 通过管理面板邀请用户。
- **会话时长：7天** — Board 成员不应该每天都要重新登录

第三章：技术架构

3.1 技术栈选择及原因

层	选择	版本	为什么选这个
前端框架	**Next.js (App Router)**	16.2.2	React 生态最成熟的全栈框架。Server Components 减少客户端 JS 体积，Server Actions 提供类型安全的后端调用，无需写 REST API
语言	**TypeScript (strict)**	5.x	类型安全，从数据库 schema 到前端组件全链路类型推导
UI 组件库	**shadcn/ui + Tailwind 4**	React 19, Tailwind 4	shadcn/ui 不是 npm 包，是复制到项目中的组件源码，完全可定制。Tailwind 4 是最新版 utility-first CSS 框架
图表	**Recharts**	3.x	React 原生图表库，声明式 API，适合 KPI 趋势图和现金流图
数据库	**Supabase (Postgres)**	—	托管的 PostgreSQL + 内置 Auth + 实时订阅 + 文件存储。一个服务搞定数据库、认证、实时推送
ORM	**Drizzle ORM**	0.45.x	类型安全的 SQL 查询构建器，schema 即代码，自动生成 migration
部署	**Vercel** (dev/staging) → **Docker** (prod)	—	Vercel 对 Next.js 零配置部署。生产环境用 Docker 部署到中国可达的云节点
PWA	**Web App Manifest**	—	让网页可以"添加到手机桌面"，接近 native app 体验
邮件通知	**Resend** (Phase 4)	—	现代邮件发送服务，API 简单，支持 React Email 模板

3.2 为什么不用其他方案

为什么不用 React Native / Flutter 做原生 App?

- 开发成本 3-5 倍。Web PWA 已经可以添加到桌面、全屏运行、推送通知
- 不需要应用商店审核和发布流程
- 一套代码适配所有设备

为什么不用传统 REST API + 单独前端?

- Next.js Server Actions 提供类型安全的 RPC 调用，不需要写 API 路由、请求/响应序列化、错误处理
- 减少一半的样板代码

为什么用 Supabase 而不是自建 Postgres?

- Supabase 内置 Auth（magic link、SSO）、实时订阅（葛总改了任务状态，Jerry 立刻看到）、RLS 策略
- 免去自己搭认证系统和 WebSocket 的工作量

- 生产环境可以迁移到自托管 Supabase（Docker 部署）

3.3 项目文件结构

```
uul-compass/
├── src/
│   ├── app/                              ← Next.js 页面（App Router 约定）
│   │   ├── (auth)/                       ← 认证相关页面（不带 dashboard 布局）
│   │   │   └── login/page.tsx            ← 登录页
│   │   ├── (dashboard)/                  ← 主应用页面（带 sidebar + topbar 布局）
│   │   │   ├── layout.tsx                ← Dashboard 壳：sidebar(desktop) + bottomnav(mobile) + topba
│   │   │   ├── page.tsx                  ← 首页仪表盘
│   │   │   ├── plan/page.tsx             ← 100天计划看板
│   │   │   ├── decisions/page.tsx        ← 决议追踪
│   │   │   └── settings/page.tsx         ← 团队管理 + 实体设置
│   │   ├── layout.tsx                    ← 根布局（字体、元数据、PWA 配置）
│   │   ├── page.tsx                      ← 根路由（重定向到 dashboard）
│   │   └── globals.css                   ← 全局主题变量（navy 配色方案）
│   │
│   ├── components/                       ← React 组件
│   │   ├── plan/                         ← 100天计划专用组件
│   │   │   ├── kanban-board.tsx          ← Kanban 看板（5列状态）
│   │   │   └── task-card.tsx             ← 单个任务卡片
│   │   ├── shared/                       ← 全局共享组件
│   │   │   ├── sidebar.tsx               ← 深色 navy 侧边栏（仅桌面端可见）
│   │   │   ├── topbar.tsx                ← 顶部导航栏（毛玻璃效果）
│   │   │   ├── bottom-nav.tsx            ← 移动端底部标签导航
│   │   │   └── nav-items.ts             ← 导航项定义（标签、图标、路由、阶段标记）
│   │   └── ui/                           ← shadcn/ui 组件库（15个已安装组件）
│   │       ├── button.tsx, card.tsx, badge.tsx, avatar.tsx
│   │       ├── input.tsx, textarea.tsx, select.tsx
│   │       ├── dialog.tsx, sheet.tsx, dropdown-menu.tsx
│   │       ├── tabs.tsx, separator.tsx, tooltip.tsx
│   │       ├── progress.tsx, scroll-area.tsx
│   │       └── （可通过 npx shadcn@latest add [name] 添加更多）
│   │
│   ├── db/                               ← 数据库层
│   │   ├── schema/                       ← Drizzle ORM Schema 定义
│   │   │   ├── enums.ts                  ← 20+ 枚举类型定义
│   │   │   ├── org.ts                    ← 组织结构：entities, offices, departments, users, contacts
│   │   │   ├── pmi.ts                    ← 100天计划：workstreams, milestones, tasks
│   │   │   ├── communication.ts          ← 活动流、评论、会议纪要、行动项
│   │   │   ├── integration.ts            ← 集成日志、审计日志
│   │   │   └── index.ts                  ← 统一导出
│   │   └── index.ts                      ← 数据库客户端初始化
│   │
│   ├── lib/                              ← 工具库
│   │   ├── supabase/
│   │   │   ├── client.ts                 ← 浏览器端 Supabase 客户端
│   │   │   ├── server.ts                 ← 服务端 Supabase 客户端
│   │   │   └── middleware.ts             ← Auth 会话中间件（刷新 token、保护路由）
│   │   └── utils.ts                      ← 工具函数（cn 类名合并等）
│   │
│   └── middleware.ts                     ← Next.js 中间件（Auth 路由保护，当前 dev 模式已禁用）
│
├── public/
│   └── manifest.json                     ← PWA 配置（应用名、图标、启动方式）
```

```
│
├── docs/
│   └── compass-handoff.md                ← 本文档
│
├── Dockerfile                            ← Docker 多阶段构建（builder + runner）
├── docker-compose.yml                    ← Docker Compose 一键部署
├── drizzle.config.ts                     ← Drizzle ORM 配置（数据库连接）
├── next.config.ts                        ← Next.js 配置（standalone output for Docker）
├── .env.local.example                    ← 环境变量模板
├── package.json                          ← 依赖列表
└── tsconfig.json                         ← TypeScript 配置
```

第四章：数据库架构（47张表，12个域）

这是 Compass 最核心的设计。数据库不只是支持 100 天计划，而是为整个物流公司的运营数据建模。

4.1 域总览

#	域	表数量	核心表	说明
1	**组织与人员**	5	entities, offices, departments, users, contacts	公司结构、办公室、部门、内部员工、外部联系人
2	**客户与CRM**	3	customers, sales_pipeline, customer_sops	客户档案、销售漏斗、客户专属 SOP
3	**承运商管理**	5	carriers, contracts, lanes, rates, performance	承运商档案、合同条款、航线、费率卡、绩效跟踪
4	**报价与定价**	3	quote_requests, quotes, quote_line_items	RFQ 管理、报价构建（origin + freight + destination 三段费用）
5	**货运生命周期**	6	shipments, containers, milestones, charges, exceptions, D&D	一票货从创建到交付的全生命周期
6	**海关与合规**	3	customs_entries, tariff_classifications, compliance_docs	报关、HS 编码、IRA 合规、FTZ
7	**财务**	9	vendors, invoices, payments, vendor_bills, GL, KPIs	应收（AR）、应付（AP）、现金流、KPI 快照
8	**3PL 仓储供应商**	2	warehouse_vendors, warehouse_bookings	第三方仓储供应商管理（LC 是供应商之一）
9	**文档**	1	documents	通用文档注册表（GDrive metadata + 链接，不存文件本身）
10	**100天计划（PMI）**	3	pmi_workstreams, pmi_milestones, pmi_tasks	工作流、里程碑、任务（含 SOP 量化 + tribal knowledge 提取）
11	**沟通**	4	activities, comments, meeting_notes, action_items	活动流、评论、会议纪要、行动项
12	**集成与审计**	3	integration_logs, audit_log, user_entity_access	外部系统同步日志、操作审计、跨实体访问权限

4.2 实体关系图

```
entities (UUL, 美航, 星航, Sage Line)
    │ 每张表都有 entity_id → 实体级数据隔离
    │
    ├── users (内部团队)
    │
    ├── customers ──→ quote_requests ──→ quotes (+ line items) ──→ shipments
    │                                        ↑                      │
    ├── carriers ──→ carrier_rates ──────────┘                      ├── containers (集装箱)
    │              → carrier_contracts (合同)                       ├── milestones (追踪节点)
    │              → carrier_lanes (航线)                           ├── charges (收支明细)
    │                                                               ├── customs_entries (报关)
    │                                                               ├── exceptions (异常)
    │                                                               ├── invoices → payments (AR)
    │                                                               ├── vendor_bills (AP)
    │                                                               ├── documents (单证)
    │                                                               └── warehouse_bookings
    │
    ├── pmi_workstreams → pmi_milestones → pmi_tasks (100天计划)
    │
    └── activities, comments (多态关联, 可评论任何实体)
```

4.3 报价与定价域（Pallet AI 集成核心）

这是连接 Compass 和 Pallet 的关键数据结构。

`carrier_rates` **表** — Pallet 的数据源：

- 按 承运商 × 航线 × 运输方式 × 集装箱类型 存费率
- 字段：carrier_id, origin_port（UN/LOCODE）, destination_port, mode（ocean_fcl/air/trucking 等）, container_type（20GP/40HC 等）
- rate, rate_currency, rate_unit（per_container/per_kg/per_cbm 等）
- effective_date, expiry_date, is_spot（即期 vs 合约价）
- 关键索引：(carrier_id, origin_port, dest_port, mode, container_type, effective_date) — 这个复合索引支撑 Pallet 的快速费率查找

`quotes` + `quote_line_items` **表** — 完整报价结构：

- 每个报价由多行 line items 组成
- line items 按 charge_category 分段：origin（起运地费用）/ freight（海运/空运费）/ destination（目的地费用）/ customs（关税）/ trucking（卡车）/ insurance / warehousing
- 每行有 buy_rate（UUL 的成本）和 sell_rate（给客户的报价），margin 自动计算
- pallet_quote_id + pallet_confidence 字段用于对接 Pallet AI 生成的报价

为什么这个设计很重要？ 目前 Marco 的费率信息在他的邮件和脑子里，没有结构化。Phase 2 的 rate card 录入工具就是为了把这些隐性知识提取出来，变成结构化数据，让 Pallet AI 能自动报价。

4.4 货运生命周期域（核心业务实体）

`shipments` **表** — 主表，一票货一条记录：

- 全运输方式：ocean_fcl, ocean_lcl, air, trucking_ftl, trucking_ltl, drayage, rail, courier, multi_modal
- 全方向：import（进口）, export（出口）, domestic（国内）, cross_trade（三角贸易）
- 状态链：draft → booked → confirmed → in_transit_origin → at_origin_port → departed → in_transit → arrived → customs_hold → customs_cleared → in_delivery → delivered → completed
- 关联关系：customer（客户）, carrier（承运商）, quote（报价）, ops_owner（操作负责人）, sales_owner（销售负责人）
- 关键单证引用：bl_number（提单号）, booking_number（订舱号）, vessel_name/voyage_number（船名/航次）, ETD/ETA/ATD/ATA（预计/实际离港到港时间）

`shipment_charges` **表** — 每票货的收支明细：

- 同样按 charge_category 分段：origin / freight / destination / customs / trucking / warehousing
- buy_amount（成本）+ sell_amount（收入）→ 每票货的利润分析
- 关联到 invoice（应收账款发票）和 vendor_bill（应付账款账单）

4.5 财务域

- **AR 链路：** invoices → invoice_line_items → payments → payment_allocations
- **AP 链路：** vendor_bills → vendor_bill_line_items
- **KPI 快照：** kpi_snapshots 表按月存储各项指标（revenue, EBITDA, DSO, shipment_count 等），用于趋势图
- **简化总账：** gl_transactions 表用于现金流追踪，不替代会计系统（QuickBooks/Xero）

4.6 已实现的 Schema（Phase 1）

当前 `src/db/schema/` 目录下已经写好的 Drizzle schema 文件：

文件	表	状态
enums.ts	20+ 枚举类型（user_role, transport_mode, shipment_status, container_type 等）	已完成
org.ts	entities, offices, departments, users, contacts, user_entity_access	已完成
pmi.ts	pmi_workstreams, pmi_milestones, pmi_tasks	已完成
communication.ts	activities, comments, meeting_notes, action_items	已完成
integration.ts	integration_logs, audit_log	已完成

尚未创建的 Schema 文件（Phase 2–4）：

文件	表	Phase
customers.ts	customers, sales_pipeline, customer_sops	Phase 2
carriers.ts	carriers, carrier_contracts, carrier_lanes, carrier_rates, carrier_performance	Phase 2
quotes.ts	quote_requests, quotes, quote_line_items	Phase 2
shipments.ts	shipments, shipment_containers, shipment_milestones, shipment_charges, shipment_exceptions, demurrage_detention	Phase 3
finance.ts	vendors, invoices, invoice_line_items, payments, payment_allocations, vendor_bills, vendor_bill_line_items, kpi_snapshots, gl_transactions	Phase 3
customs.ts	customs_entries, tariff_classifications, compliance_documents	Phase 3
warehousing.ts	warehouse_vendors, warehouse_bookings	Phase 4
documents.ts	documents	Phase 4

第五章：当前已完成的工作（Phase 1 Demo）

5.1 已搭建的5个页面

1. Dashboard 仪表盘（`/`）

- Hero 进度卡：深蓝渐变大卡片，显示 100天完成百分比（8%）、Overdue/Active/Open 三个指标
- 4 个 KPI 小卡片：Workstreams（6个活跃）、Overdue（3个需处理）、This Week（4个里程碑到期）、Blocked（1个需解决）
- 本周里程碑列表：VP Finance 招聘、Silfab 30% AR 预付款、Target Account List、企业宣传册 v2
- 最近活动流：谁做了什么，什么时候

2. 100-Day Plan 看板（`/plan`）

- Kanban 五列：To Do (11) → In Progress (7) → Blocked (1) → Review (0) → Done (3)
- 22 个真实任务，全部从 board meeting 会议纪要中提取
- 6 个 workstreams 用颜色区分：Finance(红), Operations(橙), Sales(蓝), Marketing(紫), Technology(青), Org&HR(绿)
- 每个任务卡片显示：workstream 标签、标题、优先级（Critical/High/Medium/Low + 彩色圆点）、截止日期、负责人头像

3. Decisions 决议追踪（`/decisions`）

- 4 次 board meeting 的全部决议记录：
  - 3/23 Board Sync：新销售规则（先收款再发货）、Silfab Plan A、Board 结构、CEO/COO 任命、三大业务支柱确认
  - 3/25 Operations & Cash Flow Review：VP Finance 招聘批准、$20K 营销预算、Bridge 融资、供应链金融合作
  - 3/26 Board Decision：深圳仓库扩建、Q2 招聘3名运营经理
  - 3/31 Strategy Call：物流稳定币探索、LC+Packsmith WMS 集成、Ben Fogarty 营销外包

4. Settings 设置（`/settings`）

- 团队目录：8 名成员，显示姓名、中文名、职位、角色 badge
- 实体列表：UUL Global, Meihang (美航), Star Nav (星航), Sage Line

5. Login 登录页（`/login`）

- Magic link 邮箱登录界面
- Compass logo + "Post-Merger Command Center" 副标题
- "Invitation-only. Contact Jerry for access." 提示

5.2 设计系统（当前状态）

侧边栏（仅桌面端）：

- 深色 navy 背景色（`oklch(0.18 0.04 250)`）
- 分三组导航：MAIN（Dashboard, 100-Day Plan）、OPERATIONS（Shipments, Quotes, Customers, Carriers, Finance — 后三者标 P2/P3 表示尚未开发）、SYSTEM（Decisions, Documents, Settings）
- 底部有 Day 计数器 + 绿色脉冲点

顶部导航栏：

- 固定定位 + 毛玻璃效果（backdrop-blur）
- 左：System Online 状态指示灯
- 右：Day 1 of 100 胶囊、通知铃铛（带红色 badge）、用户头像 + 姓名 + 角色

移动端底部导航：

- 4 个标签：Home, Plan, Decisions, Settings
- 活跃标签有背景高亮
- 毛玻璃效果背景

卡片样式：

- 圆角 `0.75rem`
- 微妙阴影，hover 时阴影加深 + 微微上浮
- 状态颜色系统一致：红色=Critical/Overdue, 橙色=High/Warning, 蓝色=Medium/In Progress, 绿色=Completed, 灰色=Low/Not Started

当前问题：UI 设计需要大幅提升。 虽然功能布局合理，但整体视觉效果仍然偏 generic，缺乏品牌感和高级感。需要参考 Linear、Vercel Dashboard、Notion 等级别的设计标准来重做。

5.3 当前数据状态

所有页面目前使用硬编码的 demo 数据。 还没有连接 Supabase 数据库。这意味着：

- 无法创建、编辑、删除任务
- 无法登录（登录页面只是 UI，没有后端逻辑）
- 无法实时同步（多人看到的是同样的静态数据）

接入 Supabase 后，所有这些功能都会"活"起来。

第六章：分阶段交付计划

Phase 1：基础 + PMI 追踪器（Week 1–2，当前阶段）

目标： 下次 board sync 时展示一个 live 的 100 天计划追踪器。

已完成：

- ✅ Next.js 16 项目搭建 + Tailwind 4 + shadcn/ui
- ✅ Drizzle ORM Schema（Phase 1 的 5 个文件，~15 张表）
- ✅ Supabase Auth 中间件（已写好，dev 环境暂时 bypass）
- ✅ 5 个页面的 UI + 硬编码 demo 数据
- ✅ Mobile-first 布局（sidebar + bottom nav）
- ✅ PWA manifest
- ✅ Dockerfile + docker-compose.yml

待完成（David 的任务）：

- ☐ UI 设计升级（重点：视觉品质、品牌感、交互细节）
- ☐ 部署到 Vercel（让 Jerry 和 board 可以通过 URL 看到 demo）
- ☐ 创建 Supabase 项目 + 跑 migration
- ☐ 把硬编码数据替换为 Supabase 查询（Server Actions）
- ☐ 启用 Auth（magic link 登录流程）
- ☐ 种子数据导入（4 entities, 8 users, 6 departments, 6 workstreams, 22 tasks）
- ☐ PWA 图标制作 + Service Worker

Phase 2：商业核心 — 承运商 + 报价 + 客户（Week 3–4）

目标： 打通报价链路，为 Pallet 提供结构化数据。

- 创建 Schema：customers.ts + carriers.ts + quotes.ts
- 承运商管理 UI：承运商档案、合同、航线、费率卡
- **Rate card 录入界面**（核心优先级 — 把 Marco 头脑里的费率数据结构化）
- 报价管理：RFQ 接收 → 报价构建器 → line items → 发送

- 客户 CRM：客户档案、信用条款、客户专属 SOP
- 销售 Pipeline：Kanban board
- Pallet 集成：carrier_rates API endpoint + quote webhook

Phase 3：运营 + 财务（Week 5–7）

目标： 货运全生命周期管理 + 财务可视化。

- 创建 Schema：shipments.ts + finance.ts + customs.ts
- 货运追踪器：创建 → booking → tracking → delivery → POD
- 集装箱管理 + 里程碑追踪
- AR 追踪器：Silfab 专属视图 + aging buckets（30/60/90+ 天）
- AP 管理：供应商账单
- 现金流仪表盘 + KPI 趋势图（Recharts）
- 报关追踪
- 异常管理 + 滞期费/滞港费追踪

Phase 4：完整 OS（Week 8–12）

目标： 全平台集成 + 通知 + 文档。

- 3PL 供应商管理（LC 是 preferred vendor 之一）
- 文档中心（GDrive API 自动索引 `[02] UUL Global/` 文件树）
- **双通道通知：** Email (Resend) + Telegram Group (Cloud bot)
- Weekly board digest（邮件 + TG 双推）
- Brain file sync（自动解析会议纪要 → 创建 decisions/action items）
- Google Calendar 集成
- PDF 导出（给 board deck 用）
- Org chart 可视化

第七章：David 的具体执行清单

任务 A：部署到 Vercel（优先级：最高）

目标： 获得一个公网 URL，让 Jerry 可以把 demo 分享给 board 成员看。

步骤：

1. 访问 vercel.com，用 GitHub 账号登录
2. 点击 "Import Project"，选择 `jerryshimax/uul-compass` 仓库
3. Framework Preset 会自动检测为 Next.js，直接 Deploy

4. **不需要设置任何环境变量** — demo 用的是硬编码数据，不依赖 Supabase
5. 部署成功后会得到一个 URL（类似 `uul-compass-xxx.vercel.app`）
6. 可选：设置自定义域名 `compass.uulglobal.com`
7. **用手机访问该 URL 验证移动端效果** — 这是第一优先级

任务 B：UI 设计全面升级（优先级：最高）

目标： 把 dashboard 从"能用"提升到"好看"。Jerry 对当前视觉效果不满意。

设计方向：

- 保留深色 navy sidebar — 这是品牌方向
- 参考标杆：Linear (linear.app), Vercel Dashboard, Notion, Raycast
- 航海/物流主题：navy + teal 配色已设定好，在此基础上增加质感
- 数据密度：board 成员想一眼看到大量信息，但不能杂乱

具体需要改进的区域：

1. **Dashboard 首页** — hero progress card 可以更有冲击力；KPI 卡片增加微动画或图表；milestones 列表增加进度条可视化
2. **Kanban 看板** — 任务卡片的视觉层次需要更清晰；考虑支持拖拽（未来）；点击任务弹出详情 modal
3. **排版** — 确保标题层级、字重、间距节奏一致
4. **微交互** — 平滑过渡动画、hover 状态、点击反馈、加载骨架屏
5. **空状态** — 设计没有数据时的空状态（插图 + 引导文案）
6. **颜色一致性** — 所有状态颜色（overdue=红, in progress=蓝, done=绿, blocked=红）必须全局统一

可用工具：

- Tailwind 4 — 所有 utility class
- shadcn/ui — 已安装 15 个组件，可通过 `npx shadcn@latest add [component]` 添加更多（如 table, chart, calendar, popover 等）
- Recharts — 图表/可视化
- Lucide React — 图标库（已安装）

当前色板（定义在 `src/app/globals.css` 中）：

元素	用途
Primary (Navy)	侧边栏背景、按钮、标题
Accent (Teal)	活跃状态、链接、强调
Background (浅蓝灰)	页面背景
Card (白色)	卡片背景
Destructive (红色)	逾期、关键、错误
Success (翡翠绿)	完成、正常
Warning (琥珀色)	风险、延迟

Workstream 颜色：

- Finance Integration: `#ef4444` (红)
- Operations Consolidation: `#f97316` (橙)
- Sales Engine: `#3b82f6` (蓝)
- Brand & Marketing: `#8b5cf6` (紫)
- Technology & AI: `#06b6d4` (青)
- Organization & HR: `#22c55e` (绿)

任务 C：接入 Supabase（优先级：中 — A、B 完成后）

目标： 让应用变得"活的"——可以增删改查任务、真实登录、实时同步。

步骤：

1. 访问 supabase.com，创建新项目（免费 tier 就够 demo）
2. 获取 Project URL、Anon Key、Database URL
3. 在项目根目录创建 `.env.local`（从 `.env.local.example` 复制）：

```
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
DATABASE_URL=postgresql://postgres:xxx@db.xxx.supabase.co:5432/postgres
NEXT_PUBLIC_APP_URL=https://你的vercel域名
```

4. 运行 `npx drizzle-kit push` — 这会根据 `src/db/schema/` 中的定义自动在 Supabase 中创建所有表
5. 在 `src/middleware.ts` 中取消注释 `updateSession` 调用，启用 Auth 保护
6. 写种子脚本（`src/db/seed/`）导入初始数据：4 entities, 8 users, 6 departments, 6 workstreams, 22 tasks
7. 把页面中的硬编码 demo 数据替换为 Supabase 查询（通过 Server Actions in `src/lib/actions/`）
8. 在 Vercel 项目设置中添加环境变量
9. 重新部署

任务 D：PWA 增强（优先级：中）

目标： 让 app 可以安装到手机桌面，接近 native app 体验。

步骤：

1. 制作应用图标：`icon-192.png` 和 `icon-512.png`（Compass logo，navy 背景）
2. 可选安装 `@serwist/next` 添加 Service Worker 支持
3. 配置离线缓存（已加载的页面离线可看）
4. 测试"添加到主屏幕"：iOS Safari 和 Android Chrome

第八章：环境搭建指南

8.1 前置要求

- Node.js 22+（推荐通过 nvm 安装）
- npm（注意：项目用 npm，不是 pnpm）
- Git

8.2 本地开发

```
# 克隆仓库
git clone https://github.com/jerryshimax/uul-compass.git
cd uul-compass

# 安装依赖
npm install

# 启动开发服务器
npm run dev

# 浏览器打开 http://localhost:3000
```

8.3 环境变量

复制 `.env.local.example` 到 `.env.local`，填入 Supabase 凭据：

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
DATABASE_URL=postgresql://postgres:password@db.your-project.supabase.co:5432/postgres
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

注意： 没有 `.env.local` 也能跑，只是看到的全是 demo 数据，Auth 不工作。

8.4 添加 shadcn/ui 组件

```
# 添加单个组件
npx shadcn@latest add table
npx shadcn@latest add calendar
npx shadcn@latest add popover

# 查看所有可用组件
npx shadcn@latest add
```

8.5 数据库操作

```
# 根据 schema 创建/更新数据库表
npx drizzle-kit push

# 生成 migration SQL 文件（用于生产部署）
npx drizzle-kit generate

# 打开 Drizzle Studio（可视化数据库管理器）
npx drizzle-kit studio
```

8.6 Docker 部署

```
# 构建并启动
docker compose up --build

# 应用跑在 3000 端口
# 需要设置环境变量（在 docker-compose.yml 中或通过 .env 文件）
```

Dockerfile 使用 Next.js standalone 输出的多阶段构建，最终镜像体积很小。

8.7 构建检查

```
# TypeScript 类型检查
npx tsc --noEmit

# 生产构建
npm run build

# 运行生产构建
npm start
```

第九章：设计参考与竞品对标

在设计升级时，可参考以下产品的设计语言：

产品	参考点
Linear (linear.app)	侧边栏设计、键盘快捷键、任务卡片交互、极简但高信息密度
Vercel Dashboard	数据可视化、KPI 卡片、部署状态指示器、暗色主题
Notion	看板视图、页面内编辑、多视图切换（表格/看板/时间线）
Airplane.dev	内部工具仪表盘设计、表格组件、权限系统 UI
Flexport Dashboard	货运追踪 UI、货运状态时间线、地图可视化（物流行业竞品参考）

第十章：关键联系人

人员	角色	联系方式	说明
Jerry Shi	项目负责人	Telegram、Email	最终决策者，在亚洲时区
David Wu	工程师	—	负责部署、设计、技术实现，在北美时区
Alic Ge (葛总)	Board 成员，关键干系人	Telegram（仅手机）	**只用手机，任何 UI 变更都必须在手机上验证**
Jason Likens	CEO US	Email	使用系统，代表美国团队的体验反馈

第十一章：跨时区协作机制（Follow the Sun）

Jerry 在亚洲（UTC+8），David 在北美（UTC−5 ~ UTC−8）。时差 12–16 小时。这不是劣势——如果交接做得好，等于 **24 小时不间断开发**。Jerry 下班时 David 刚上班，David 下班时 Jerry 刚上班。

核心原则：**零会议异步协作**。所有上下文通过文件传递，不靠口头沟通。目标是每周不超过 1 次同步通话。

11.1 HANDOFF.md — 每日交接文件

仓库根目录有一个 `HANDOFF.md` 文件，它是**滚动更新的操作日志**，不是文档。每个人在结束工作时更新自己的部分，保留对方的部分不动。

文件结构：

```
# Compass Handoff

> Last updated: 2026-04-02 23:45 UTC+8 by Jerry

## Status: ON TRACK | BLOCKED | NEEDS DECISION

## Current Sprint Focus
本周目标的一句话描述。

---

## Jerry's Last Session (日期, 时区)

### Done
- 完成了什么（具体，可验证）

### In Progress
- 正在做什么，进度到哪里了
- 阻塞点（如果有）

### For David
1. **最高优先：** 具体任务描述（不是"做 UI"，而是"实现任务卡片点击弹出 modal，参考 Linear 的 issue detail 面
2. **次优先：** ...
3. **注意事项：** 特定文件的陷阱或变更

### Open Questions
- 需要讨论但不紧急的问题

---

## David's Last Session (日期, 时区)
（同样结构）
```

为什么这样设计：

- **"For David" / "For Jerry" 是核心** — 它回答"下一个人该做什么"。编号、有优先级、具体到可执行
- **每个人覆盖自己的部分** — 意味着你总能同时看到：对方上次做了什么 + 给你留了什么任务
- **Status 一行** — 一眼判断是否有阻塞。如果写了 BLOCKED，对方上班后第一件事处理
- **不是追加日志** — 老的历史在 `git log HANDOFF.md` 里，这个文件永远只有最新状态，不超过 80 行
- **Sprint Focus** — 让两个人对齐这周的目标，不需要开 standup

11.2 每日节奏

Jerry 下班前（约 23:00–00:00 UTC+8 = David 上午 10:00–11:00 EST）

1. Push 所有 work-in-progress 分支（即使未完成也要推，防止代码只在本地）
2. 为准备好的工作开 PR。如果需要 David review，标 `needs-review` label
3. **更新 HANDOFF.md** — 填写 Done、In Progress、For David。Commit 并 push 到 main
4. 如果有紧急阻塞，通过 Telegram 发消息给 David。否则 HANDOFF.md 就够了

David 上班后（约 09:00 EST = Jerry 晚上 22:00 UTC+8）

1. `git pull main` 。**打开 HANDOFF.md** — 这就是你的 standup
2. 看有没有标了 `needs-review` 的 PR。有的话前 30 分钟先 review
3. 按 "For David" 列表的优先级顺序工作
4. 下班前更新 HANDOFF.md（自己的部分），push

重叠窗口（可选的同步时间）

Jerry 的上午 08:00–10:00 UTC+8 = David 的晚上 19:00–21:00 EST。这是唯一可能同时在线的窗口。仅在需要时使用（见 11.4 升级机制）。大多数天不需要。

11.3 Git 工作流

分支命名

```
jerry/<功能>     例: jerry/kanban-dnd
david/<功能>     例: david/ui-upgrade
```

用名字前缀防止冲突，一眼看出是谁的分支。

PR 规范

标题格式： `[区域] 简短描述`

- `[ui] task card detail modal`
- `[db] supabase client + migration setup`
- `[infra] vercel deployment config`

PR 正文模板（已配置在 `.github/PULL_REQUEST_TEMPLATE.md` ）：

```
## What （做了什么）
## Why （为什么做）
## How to Test （如何验证）
## Screenshots （UI 变更必须附截图：手机 + 桌面）
## Handoff Notes （交接备注）
```

异步 Review 协议

核心问题：David 开 PR 时 Jerry 在睡觉，PR 会空等 12 小时。解决方案——**信任制自合并 + 事后 review**：

场景	操作
纯 UI 变更、增量功能、有自信	**自行合并**。在 HANDOFF.md 里注明合了什么
结构性变更：schema 改动、auth 逻辑、middleware、共享工具	标 `needs-review`，在 HANDOFF.md "For [name]" 中注明。**不要自行合并**
PR 评论来回超过 3 轮无法解决	升级为 15 分钟同步通话

关键规则：

- PR 控制在 300 行以内。大 PR 会阻塞一个完整的时区周期
- 永远不要 force push 到 main
- Main 分支必须始终可部署

冲突预防

两个人不要同时改同一个文件。HANDOFF.md 的 "For David" / "For Jerry" 部分隐式分配了工作区域。如果 Jerry 说"我在改 `kanban-board.tsx`"，David 就知道不要碰这个文件。

11.4 升级机制：什么时候打破异步

异步工作在以下情况下需要切换到同步（Telegram 通话或消息）：

1. **HANDOFF.md 写了 `Status: BLOCKED`** — 阻塞点需要对方输入，不能自己解决
2. **架构层面的分歧** — PR review 评论超过 3 轮没共识
3. **共享状态损坏** — 数据库 migration 冲突、main 分支挂了、Vercel 部署失败且需要双方 debug
4. **紧急外部 deadline** — Jerry 明天要给 board 演示，某功能没跑通
5. **需求不清楚** — David 看了 "For David" 但真的不理解要做什么（这意味着 Jerry 的交接写得太模糊——需要改进流程）

升级渠道： Telegram（通过 Cloud bot 或直接消息）。不用邮件，不用 GitHub Issues。Telegram 对紧急事项够及时。

目标：每周不超过 1 次同步通话。 如果通话频率更高，说明 HANDOFF.md 的笔记不够详细。

11.5 自动化工具

GitHub Action：交接文件过期提醒

已配置在 `.github/workflows/handoff-check.yml` 。每天 UTC 中午检查一次，如果 HANDOFF.md 超过 36 小时没更新，自动创建一个 GitHub Issue 提醒。

这抓的是"有人忘了更新交接日志"的场景。

PR 模板

已配置在 `.github/PULL_REQUEST_TEMPLATE.md` 。每次开 PR 时自动填充模板，确保截图和交接备注不被遗漏。

11.6 不用什么

不用	原因
Jira / Linear / Notion Board	2 个人不需要重型项目管理工具。GitHub Issues + HANDOFF.md 够了
Slack / Discord	Telegram 已经在用，不增加沟通渠道
每日 standup bot	HANDOFF.md 就是 standup，由人写的，有上下文
复杂 CI/CD	Vercel 从 main 自动部署就够了

11.7 总结：整个机制就是这些

工件	说明
`HANDOFF.md`	仓库根目录，每日滚动更新的交接日志
`.github/PULL_REQUEST_TEMPLATE.md`	PR 模板，确保截图和交接备注
`.github/workflows/handoff-check.yml`	过期提醒，36h 未更新自动创建 Issue
分支前缀 `jerry/` `david/`	防止冲突，一眼看出是谁的
Telegram	仅紧急升级用

五个工件，零会议，最小开销。

附录：关键设计约束提醒

1. **葛总只用手机** — 这不是"也要支持移动端"，而是"移动端是主要使用场景"。每个设计决策都要先想手机上怎么用。

2. **英文 UI，中英文内容** — 界面标签、按钮、导航全英文。但任务标题、描述、评论可以写中文。
3. **PWA 安装** — 要能添加到手机桌面，打开后全屏运行，不显示浏览器地址栏。
4. **Day 1 已经开始** — 4月1日是 Day 1，时间在走。系统需要尽快上线让 board 开始用。
5. **数据安全** — 财务数据只有特定角色可见。RLS 在数据库层面强制执行，不是前端隐藏。

本文档生成于 2026年4月1日。完整架构细节详见 Drizzle schema 文件 `src/db/schema/` 和计划文件。